

BÀI 04: ROUTING VÀ CONTROLLER TRONG NESTJS

Mục tiêu bài 04: Bài này sẽ giới thiệu đến các bạn về routing và controller trong Nestjs, giới thiệu đến các bạn về Route động và tham số, xử lý query parameter và xử lý body với @Post.

4.1. ROUTING TRONG NESTJS

Trong NestJS, **Routing** là cơ chế ánh xạ (map) giữa đường dẫn URL và hành động xử lý tương ứng trong hệ thống.

Mỗi khi người dùng gửi một yêu cầu HTTP (GET, POST, PUT, DELETE...), NestJS sử dụng **router** để định tuyến (route) yêu cầu đó tới phương thức tương ứng của **Controller**.

Ví dụ về một routing:

```
@Get(hello)          //Phương thức sử dụng là Get

getHello() {          //Tên phương thức

    return 'Xin chào mọi người';

}
```

- Lúc này nếu các bạn gõ vào đường dẫn website là: **localhost:8080/hello** thì chương trình sẽ thực hiện phương thức getHello()

4.2. CONTROLLER TRONG NESTJS

Controller là thành phần chịu trách nhiệm tiếp nhận các **request** từ **client**, xử lý sơ bộ (hoặc chuyển giao xử lý cho **Service**), sau đó trả về **response** tương ứng cho **client**.

Trong NestJS, mỗi **Controller** là một class có thể chứa nhiều phương thức xử lý các **HTTP** method như GET, POST, PUT, DELETE,...

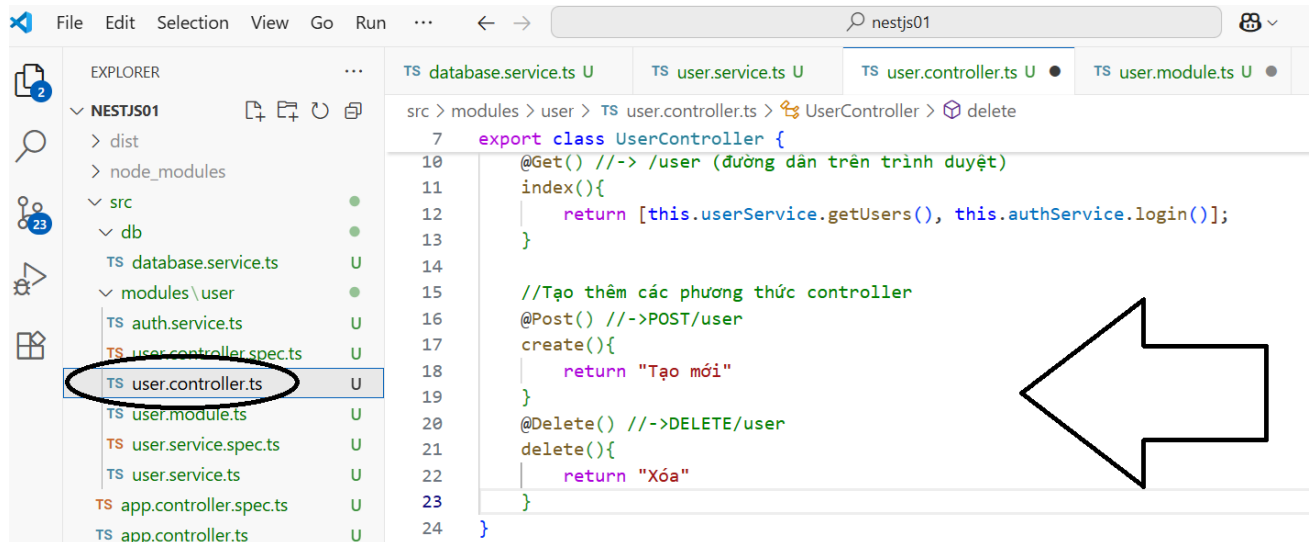
NestJS sử dụng decorator như @Controller(), @Get(), @Post()... để định nghĩa các route và hành vi của chúng.

HTTP Method	Decorator	Ý nghĩa
GET	@Get()	Lấy dữ liệu
POST	@Post()	Thêm mới dữ liệu
PUT	@Put()	Cập nhật toàn bộ dữ liệu
PATCH	@Patch()	Cập nhật một phần dữ liệu
DELETE	@Delete()	Xóa dữ liệu

4.3. VÍ DỤ VỀ SỬ DỤNG CONTROLLER TRONG NESTJS

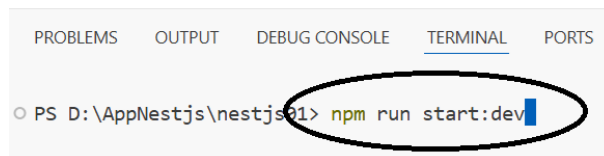
Ví dụ 4.1: Tạo các Controller với các phương thức khác nhau

- Các bạn mở lại dự án đã tạo ở Bài 3 (*Bài trước*)
- Mở tập tin **user.controller.ts** và viết thêm các phương thức **Post ()**, **Delete ()** như bên dưới:



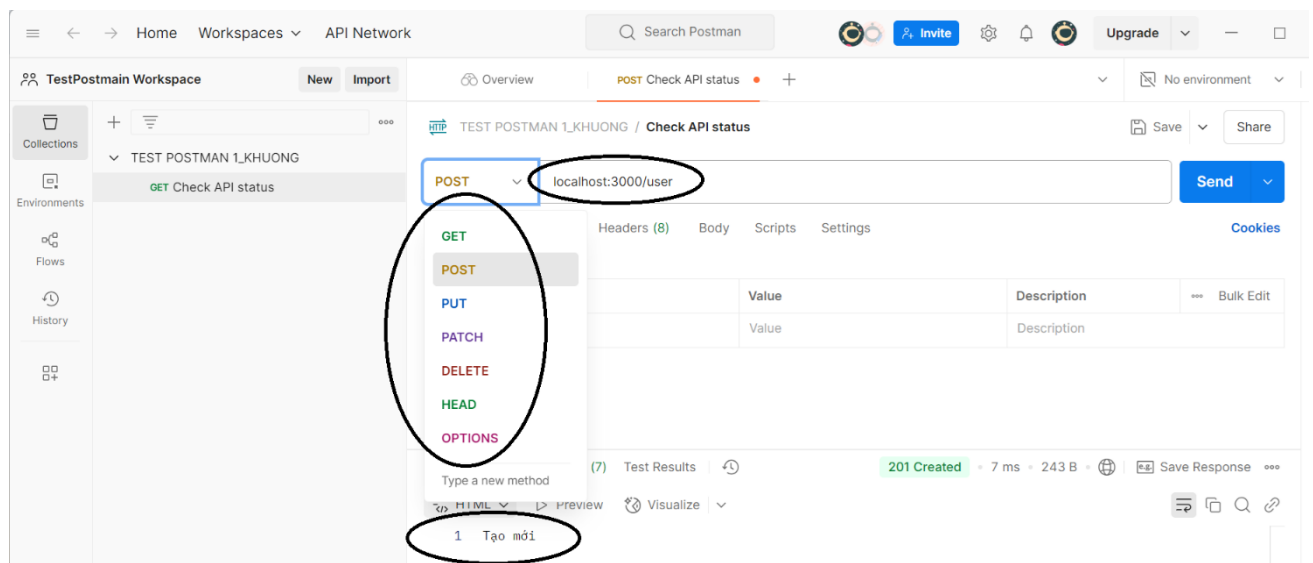
```
src > modules > user > TS user.controller.ts > UserController > delete
7  export class UserController {
10  @Get() //-> /user (đường dẫn trên trình duyệt)
11  index(){
12      return [this.userService.getUsers(), this.authService.login()];
13  }
14
15  //Tạo thêm các phương thức controller
16  @Post() //->POST/user
17  create(){
18      return "Tạo mới"
19  }
20  @Delete() //->DELETE/user
21  delete(){
22      return "Xóa"
23  }
24 }
```

- Mở **Terminal** gõ lệnh biên dịch chương trình: **npm run start:dev**



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\AppNestjs\nestjs01> npm run start:dev
```

- Để kiểm tra các phương thức khác nhau trong chương trình, các bạn dùng phần mềm **Postman** để kiểm tra. Mở chương trình **Postman**, gõ đường dẫn và chọn phương thức tương ứng sẽ cho ra kết quả.

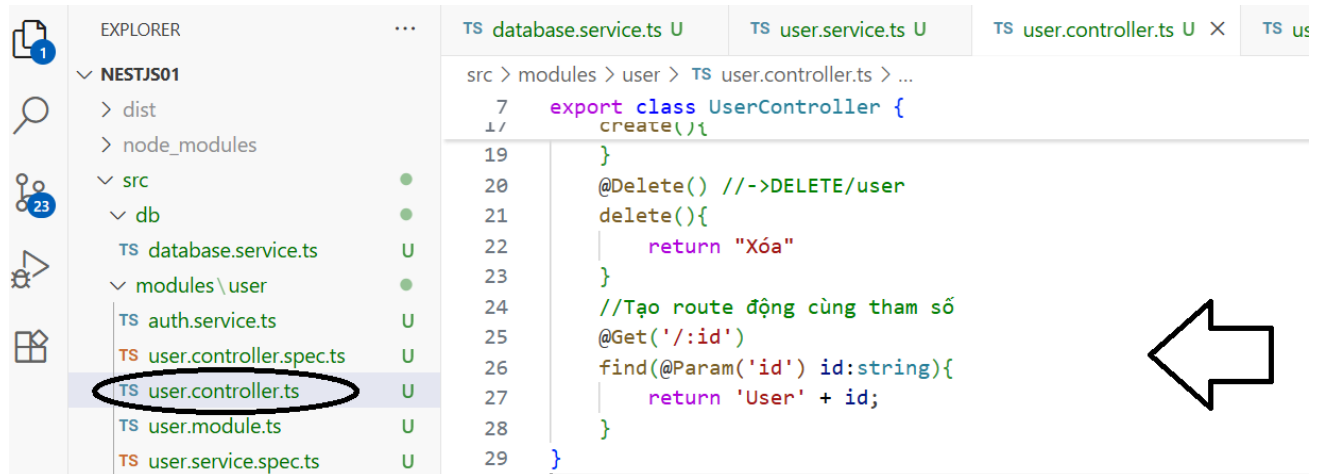


4.4. ĐỊNH NGHĨA ROUTE ĐỘNG VÀ THAM SỐ (PARAMS)

Route động cho phép truyền tham số trên URL, ví dụ: **/user/:id**

Ví dụ 4.2: Tạo Route động

- Mở tập tin **user.controller.ts** và viết thêm đoạn code như bên dưới để tạo **Route** động cùng tham số:

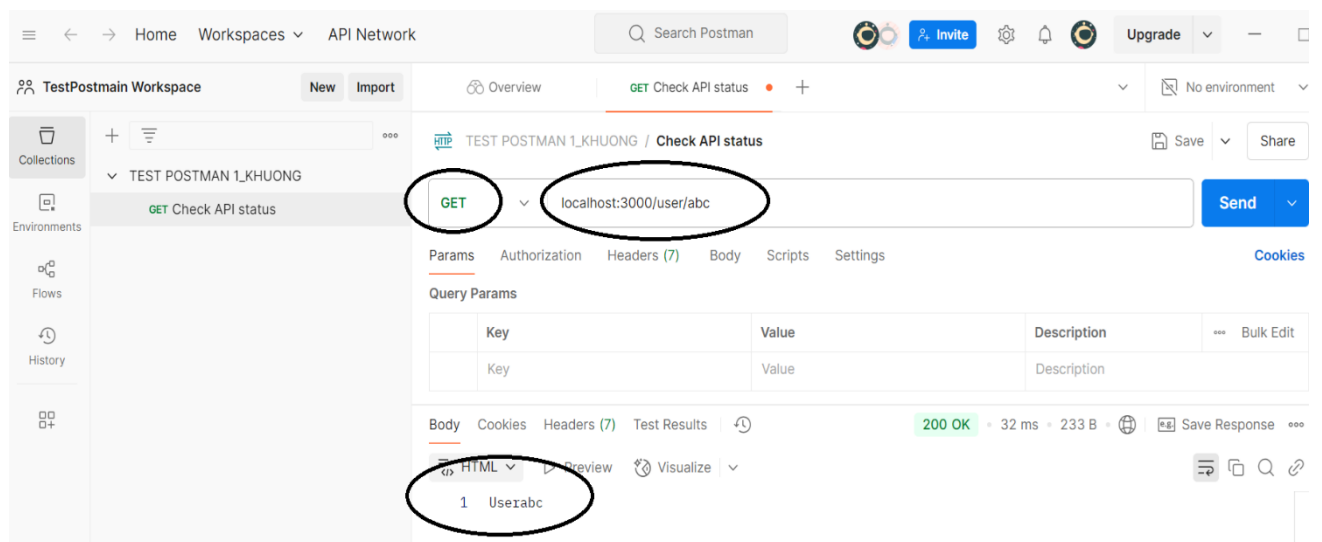


```

src > modules > user > TS user.controller.ts > ...
7   export class UserController {
8     create() {
9       // ...
10    }
11  }
12
13  @Delete() //->DELETE/user
14  delete() {
15    // ...
16    return "Xóa"
17  }
18
19  //Tạo route động cùng tham số
20  @Get('/:id')
21  find(@Param('id') id:string){
22    return 'User' + id;
23  }
24
25  }
  
```

- Lưu tập tin và biên dịch chương trình lại: **npm run start:dev**

- Mở chương trình **Postman** để kiểm tra, tùy thuộc vào giá trị nhập vào **Route** (**localhost:3000/user/abc**) sẽ thấy kết quả hiển thị tương ứng.

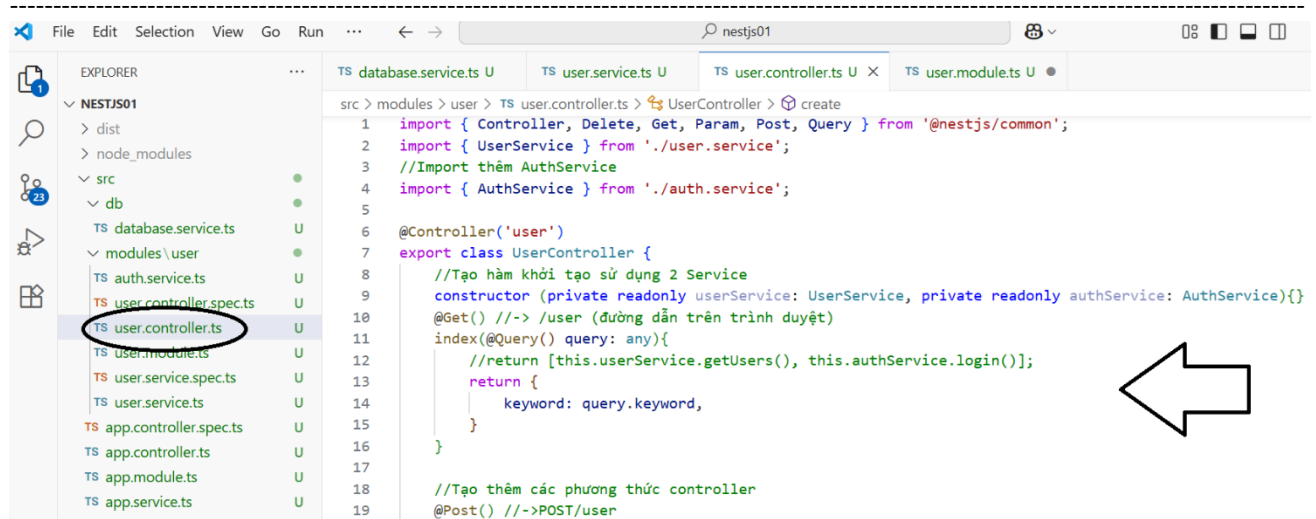


4.5. XỬ LÝ QUERY PARAMETERS

Ví dụ 4.3: Xử lý Query Parameters

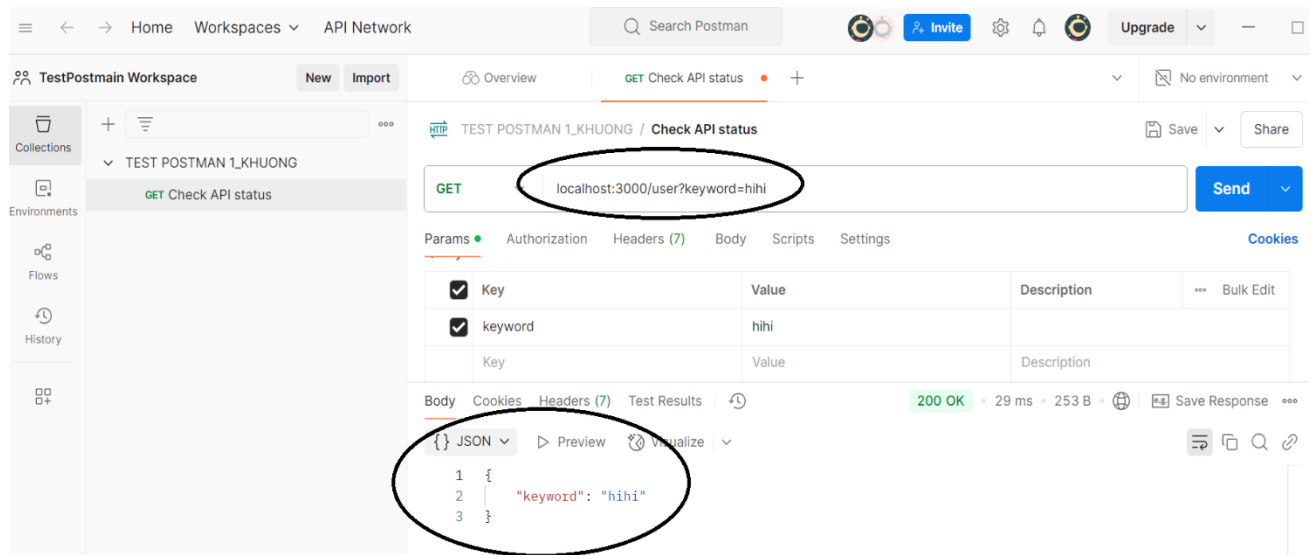
- Mở tập tin **user.controller.ts** và sửa lại code cho phương thức **index()** như bên dưới để xử lý query:

Bài 04: Routing và Controller trong Nestjs



- Lưu tập tin và biên dịch chương trình lại: **npm run start:dev**

- Mở chương trình **Postman** để kiểm tra, nhập query (**localhost:3000/user?keyword=hihi**) sẽ cho kết quả:

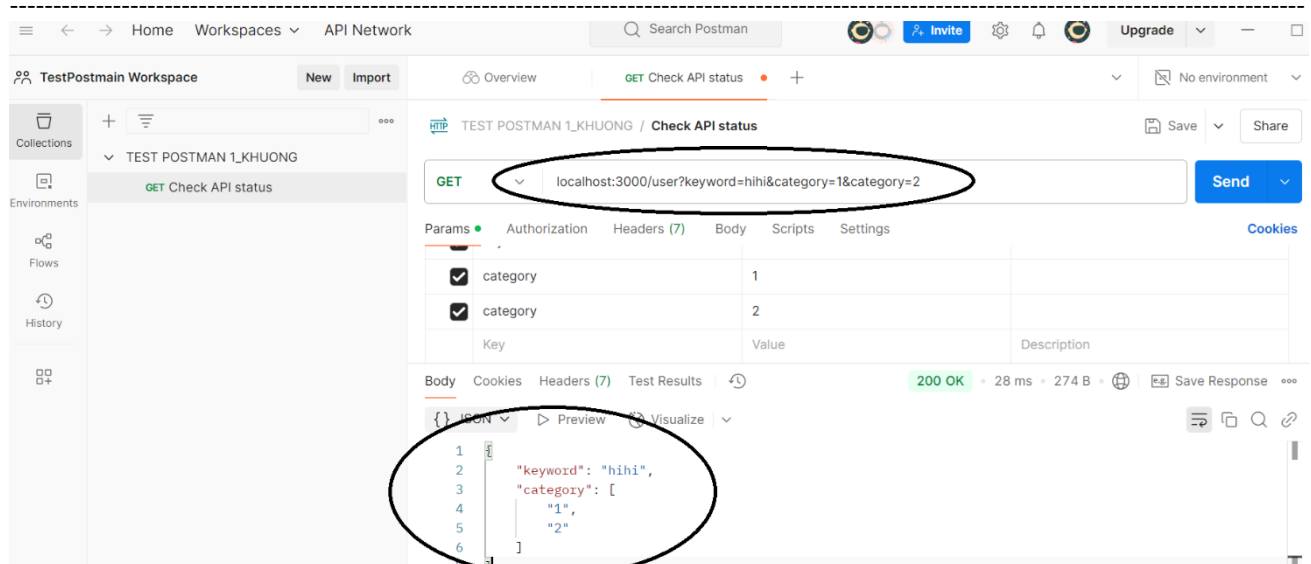


- Chúng ta cũng có thể tạo nhiều query cùng lúc, ví dụ chúng ta sửa lại code cho phương thức `index ( )` như sau:

```
index(@Query() query: any){
  //return [this.userService getUsers(), this.authService.login()];
  return {
    keyword: query.keyword,
    category: query.category,
  }
}
```

- Biên dịch chương trình và mở **Postman** để kiểm tra nhiều query (**localhost:3000/user?keyword=hihi&category=1&category=2**) như sau sẽ cho kết quả:

Bài 04: Routing và Controller trong Nestjs

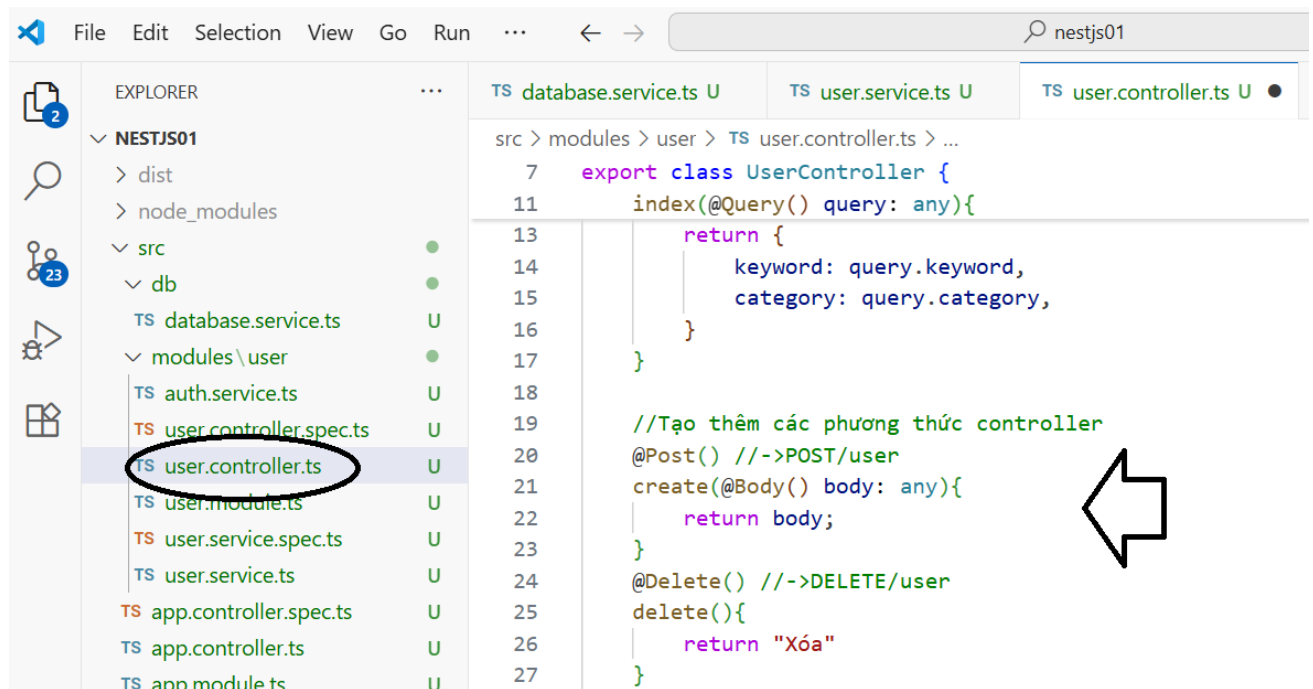


4.6. XỬ LÝ BODY VỚI @POST

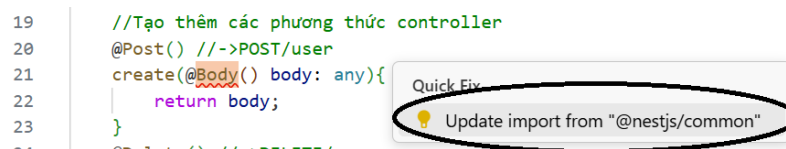
Đối với **HTTP POST**, dữ liệu thường được truyền trong body của request (truyền từ client lên server).

Ví dụ 4.4: Xử lý Body

- Mở tập tin **user.controller.ts** và sửa lại code cho phương thức **@Post()** như bên dưới để xử lý body:



- Nếu có lỗi thì rê chuột vào chỗ lỗi chọn **Quick fix** (để import thêm thư viện)



- Lưu tập tin và biên dịch chương trình lại: **npm run start:dev**
- Mở chương trình **Postman** để kiểm tra, chọn thẻ **Body**, chọn **Raw**, gõ nội dung vào **Raw** và **Send** sẽ thấy nội dung trong **Body** từ **Client** sẽ được gửi lên **Server**.

